OS SIMULATOR PRO

Technical Report: Challenges Faced Using the STAR Format

Submitted By:
Afifa Zain Apurba (1912310642)
Course: 323 Operating Systems
Project: OS Simulator Pro
File Location: *https://github.com/AfifaZain/OS*
Website: *https://afifazain.github.io/OS/*

1. Project Overview

OS Simulator Pro is a browser-based simulator for operating system concepts, built with HTML, CSS, and JavaScript. It runs from a single index.html file with no installation and no server required.

The project covers seven modules: CPU Scheduling, Memory Management, Page Replacement, Disk Scheduling, Process Management, Deadlock Detection, and a simulated Terminal. Together they implement more than twenty algorithms.

This report documents five technical challenges encountered during development. Each challenge follows the STAR format: Situation describes what went wrong, Task describes what the code was trying to do, Action describes the fix, and Result describes what changed.

2. Challenges Faced (STAR Format)

Challenge 1: Implementing Preemptive CPU Scheduling Algorithms

Preemptive scheduling was harder than expected. SRTF and Preemptive Priority require the scheduler to interrupt a running process the moment a shorter or higher-priority process arrives, which means re-evaluating the ready queue at every simulated time unit.

Early builds had two consistent bugs. First, processes kept running past their preemption points. Second, waiting times came out wrong because the code confused original burst times with remaining burst times. The Gantt chart was also cluttered: it recorded one block per time unit instead of merging consecutive runs of the same process into a single segment.

The CPU module had to support six algorithms: FCFS, SJF (non-preemptive), SRTF (preemptive), Priority non-preemptive, Priority preemptive, and Round Robin. Each one needed to produce a color-coded Gantt chart and three per-process metrics: Turnaround Time (TAT), Waiting Time (WT), and Response Time (RT).

The scheduler was rewritten to step through time one unit at a time. Each tick re-evaluated the ready queue from scratch. Remaining burst time was stored in a separate field so the original burst time stayed intact for metric calculations.

Gantt chart generation was also fixed. Rather than writing one entry per tick, the code now merges consecutive ticks belonging to the same process into a single block. Context switches are detected and logged as their own entries. This made the chart readable at a glance.

All six algorithms now produce correct output. Preemptions happen at the right moments, response times reflect the first actual CPU grant per process, and the Gantt chart reads cleanly without clutter. The metric tables match hand-calculated reference values for every test case run.

Challenge 2: Visualizing the Memory Map Dynamically

The memory manager needed a live block diagram of physical RAM that updated after every allocation and deallocation. The early version had three problems: blocks overlapped, free holes were not drawn to scale, and the compaction button rearranged the display without touching the underlying data structure. That last bug meant the visual and the actual state disagreed after every compaction.

The module had to support four allocation strategies: First Fit, Best Fit, Worst Fit, and Next Fit. After each operation, the map had to show allocated blocks colored by process and free holes in grey. Statistics below the map included used memory, free memory, fragmentation percentage, and process count.

Memory was modelled as a sorted array of segments. Each segment stores a start address, a size, and an occupant field that is null when the segment is free. The visual map renders by iterating this array and drawing CSS blocks sized proportionally to each segment's size relative to total RAM.
Fragmentation is calculated as the number of free holes divided by total free space, which captures external fragmentation rather than just raw free bytes. Compaction now physically reorders the array, shifts all allocated segments to the lowest addresses, rebuilds the free hole at the top, and then re-renders from the updated array. Display and data stay in sync.

The map updates after every operation. Fragmentation figures are correct. Compaction works: allocated blocks move together, the free space consolidates, and the display reflects the new state without any inconsistencies. All four strategies place processes without overlapping blocks.

Challenge 3: Implementing the Optimal Page Replacement Algorithm

The Optimal (Belady's) algorithm evicts whichever page in the current frame set will not be needed again for the longest time. Implementing that look-ahead in JavaScript introduced off-by-one errors. Pages were being evicted at the wrong step, so the page fault count came out higher than the theoretical minimum, which defeated the point of having an Optimal baseline.

The paging module needed four algorithms: FIFO, LRU, Optimal, and Clock. Each had to produce a step-by-step frame trace with a fault or hit flag per step, plus a final page fault count, hit count, and hit ratio. The Optimal algorithm specifically had to match the proven minimum fault count so the other three could be benchmarked against it.

For each eviction decision, the code scans forward through the remaining reference string from the current position. It checks every page currently in the frame set and records the index of its next appearance. Pages with no future reference get an infinity priority and are evicted first. Among the rest, the one with the farthest next use is chosen.
The trace recording was also corrected. Each step now captures the full frame state after the reference is processed, not before, which fixed the off-by-one issue.

The Optimal algorithm now hits the correct minimum fault count on every reference string tested. The step-by-step trace matches hand-traced reference tables. Students can now run the same string through FIFO and LRU and directly see how much worse each performs compared to the theoretical best.

Challenge 4: Drawing the Disk Head Movement on Canvas

The disk scheduler draws the head's path across tracks as a line graph on an HTML5 Canvas. The first versions had scaling problems. Lines ran off the canvas edge when disk sizes were large, axis labels overlapped when request queues were long, and the scale changed inconsistently between runs with different parameters.

Six algorithms needed canvas support: FCFS, SSTF, SCAN, C-SCAN, LOOK, and C-LOOK. Each had to show the head's full movement path, label each stop, and display total seek distance and average seek time below the chart.

A normalization function maps track numbers to canvas X-coordinates. It takes the disk size as input and applies a fixed left and right padding, so no track position can ever fall outside the drawable area. Y-coordinates are assigned sequentially based on visit order, with spacing calculated from canvas height divided by the number of stops.
The canvas renders in two passes. The first pass draws the grid and axis labels. The second draws the movement path as connected line segments with a filled circle at each stop. This layering keeps the path visually on top of the grid without needing z-index tricks.

The visualization works across all six algorithms, various disk sizes, and queues of different lengths. Labels do not overlap. The chart makes the difference between FCFS and SCAN immediately visible: FCFS zigzags across the disk while SCAN sweeps back and forth in a predictable pattern.

Challenge 5: Implementing the Banker's Algorithm and Resource Allocation Graph

The Banker's Algorithm works by checking whether a system state is safe: can all processes finish given the current allocation and the remaining available resources? Implementing the need matrix (Need = Max minus Allocation), tracking available resources across iterations, and detecting safe sequences required careful handling of 2D arrays in JavaScript.
Early versions had two failure modes. Sometimes the code reported a safe sequence when none existed. Other times it failed to find a sequence that did exist. The Resource Allocation Graph was also a separate problem: drawing SVG arrows between process and resource nodes at the right angles required coordinate math that the first attempt got wrong.

The module had to let users set the number of processes and resource types, enter allocation and maximum matrices, and run the algorithm to get a safety verdict. It also had to render a Resource Allocation Graph showing allocation edges (resource to process) and request edges (process to resource) as labeled arrows.

The algorithm uses a finish array initialized to false for each process and a work array initialized to the current available resources. The main loop searches for any unfinished process whose need vector fits within work. When found, its allocation is added back to work, it is marked finished, and its name is appended to the safe sequence. If the loop ends with any process unfinished, the system is unsafe.

For the RAG, process nodes and resource nodes are placed in two columns using absolute positioning inside a fixed-size container. SVG arrows are drawn by reading the rendered positions of each node and calculating the correct start and end coordinates for each edge. Allocation edges and request edges use different colors to stay visually distinct.

The algorithm correctly identifies safe and unsafe states across all configurations tested. When the state is safe, it outputs the sequence. When it is not, it lists which processes could not be scheduled. The RAG renders with correct directional arrows and updates whenever the user changes the input matrices.

3. Summary of Challenges

#	Challenge	Action Taken	Outcome
1	Preemptive CPU scheduling logic	Time-unit simulation with per-tick queue re-evaluation	All six algorithms produce correct Gantt charts and metrics
2	Dynamic memory map visualization	Segment array model with proportional CSS block rendering	Live memory map with working compaction and correct fragmentation
3	Optimal page replacement look-ahead	Forward scan selecting the farthest-next-use eviction	Theoretical minimum page faults on all tested reference strings
4	Disk head canvas drawing	Normalized coordinate mapping with two-pass layered drawing	Correctly scaled path for all six algorithms and all input sizes
5	Banker's algorithm and RAG rendering	Finish/work array safety check with SVG coordinate math	Correct safety detection and directional RAG for all configurations

4. Conclusion

Each module in this project required two separate problems to be solved: getting the algorithm right, and getting the output readable. Those two goals pulled in different directions more than expected. A correct algorithm that produces a cluttered chart or a mislabeled canvas is not much use to a student trying to understand why SCAN outperforms FCFS.

The five challenges here cover the cases where that gap was hardest to close: preemptive scheduling logic that kept producing wrong waiting times, a memory map that diverged from its own data after compaction, an Optimal algorithm that was losing to the theoretical minimum it was supposed to define, a canvas that could not scale to different inputs, and a safety checker that reported the wrong verdict.

Fixing each one meant rethinking the data structure first and the rendering second. The segment array for memory, the finish/work arrays for Banker's, the normalized coordinate function for disk, and the forward-scan for Optimal were all data-level changes that made the display problems simpler to solve.

The result is a simulator that runs twenty-plus OS algorithms in a browser, with no setup, and produces visual output that a student can read without already knowing the answer. That was the goal from the start.